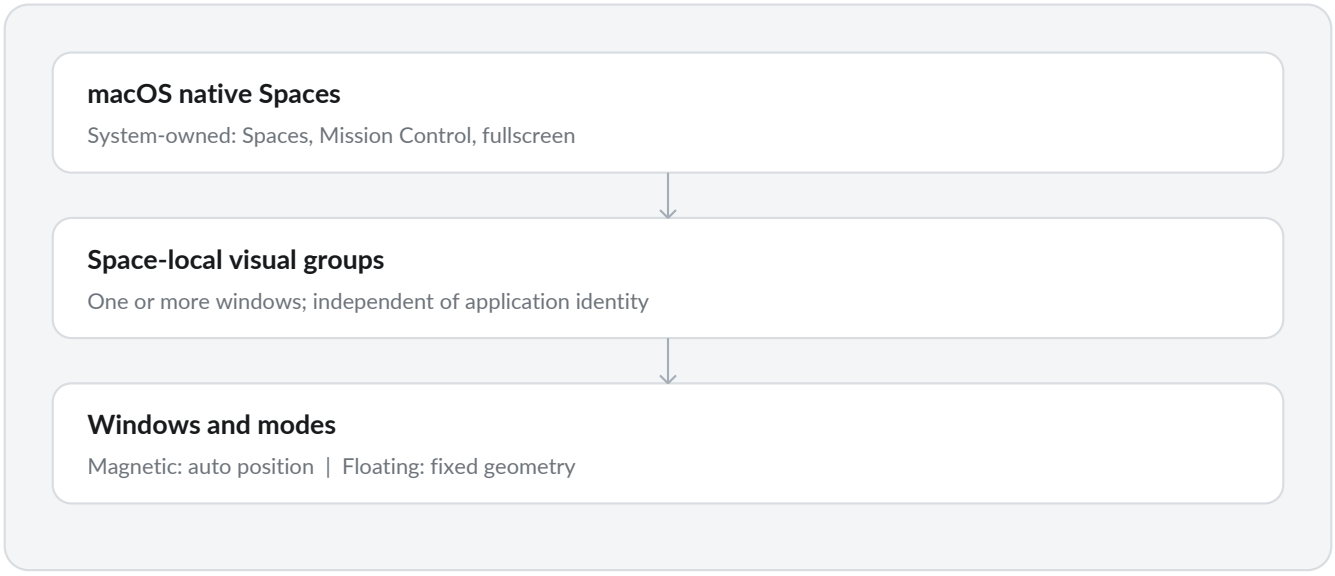
PRODUCT DEFINITION / DRAFT 0.1

Magnetic Window Manager

A native-first focus, navigation and window-positioning system for macOS.

THE PRODUCT IDEA

Preserve the macOS desktop. Improve how its windows are discovered, focused and arranged. Let magnetic windows move intelligently, let floating windows stay where the user placed them, and never make the window manager feel like a separate desktop environment.



Status

This specification records conventions agreed during design discussion. Items marked Proposed or Open remain unvalidated. It is not a claim that all platform behaviors are currently implementable.

SOURCE MATERIAL · Design conversation and original hand-drawn concept included in appendix.

01 / NORTH STAR

Product principles

A smaller product, executed exceptionally well, is preferable to an elaborate manager that disrupts native macOS behavior.

Principle	Product consequence
Native by default	Use genuine macOS windows and Spaces; preserve Mission Control, Dock, Cmd+Tab and normal fullscreen.
Invisible until needed	No permanent tiling frames, bespoke title bars, extra desktop panels or persistent controls.
Window-first interaction	Focus, navigation, repositioning and simple snapping matter more than explicit layout trees.
Predictable ownership	Magnetic mode owns automatic position; floating mode leaves user-defined geometry untouched.
No surprise resizing	Automatic positioning never changes a window's dimensions. Resizing requires deliberate input.
Graceful degradation	If a macOS API or gesture is unreliable, retain robust keyboard/native behavior rather than force an imitation.

Explicit non-goals for v1

- Not a Niri port, continuous infinite scrolling compositor or full Mission Control replacement.
- Not a replacement for native Spaces; not a virtual-workspace manager.
- Not a rigid BSP tree or a requirement that every window fill an assigned tile.
- No separate pixel-by-pixel resize shortcuts, advanced rules engine, or custom full-screen animations.
- No finalized Option+M action; the shortcut remains reserved and undefined.

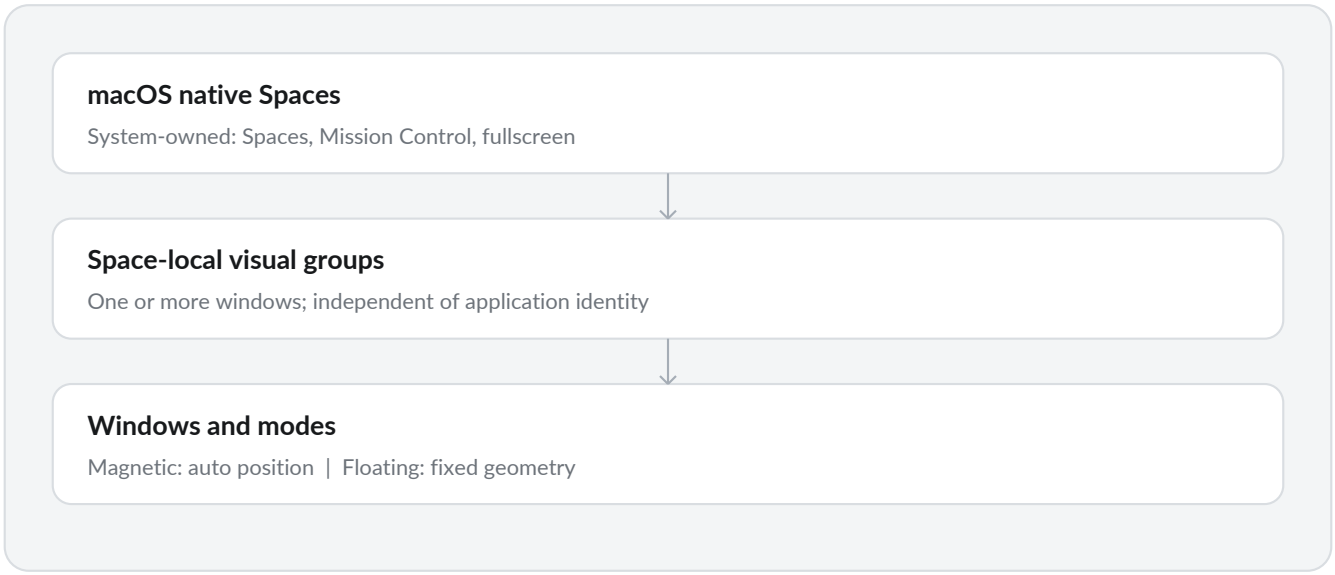
Acceptance philosophy

Native behaviors must continue to work without visual jumps, lost windows or unnecessary repositioning. A quick, reliable transition is preferable to a sophisticated but unstable animation.

02 / CONCEPTUAL MODEL

Windows, visual groups and Spaces

A visual group represents the windows jointly presented in one arrangement; it does not represent an application.



Core entities

Entity	Definition
Native Space	An actual macOS Space, owned by the operating system.
Window	A real application window tracked for focus and, where supported, manipulation.
Visual group	One or more windows that share a co-visible arrangement; e.g., a left/right split is one group.
Window stack	An ordered navigation sequence across windows in the current Space, including grouped and floating windows.
Focused window	The window currently receiving keyboard input; logically distinct from group and overlay selection.
Magnetic / Floating	Per-window modes that determine whether automatic repositioning is permitted.

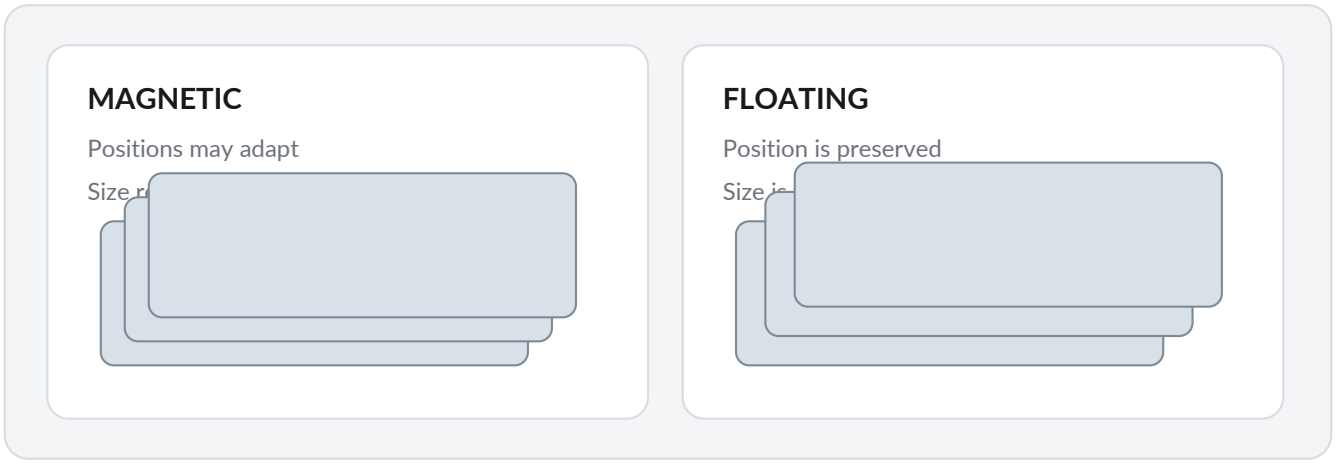
Identity and ordering

- A group can contain windows from different apps; one app can have windows in different groups.
- Group jumping and individual-window navigation are separate commands with separate semantics.
- Each Space maintains its own ordering, group structure, focus history and mode state as far as the platform allows.
- For a group entry, restore the last-focused member by default (proposed policy).

03 / GEOMETRY OWNERSHIP

Magnetic vs floating

The fundamental contract is about what can move automatically - not about enforcing a tiled size.



Operation	Magnetic window	Floating window
Automatic position change	Allowed	Never
Automatic size change	Never	Never
Manual move / resize	Allowed	Allowed
Explicit snap to region	Allowed	Allowed
Stack focus / Z-order	Participates	Participates
Native app/window controls	Preserved	Preserved

Mode transitions

- Magnetic → Floating: freeze the current rectangle as the user-owned position and size.
- Floating → Magnetic: retain the current dimensions, rejoin managed placement, and permit repositioning.
- Either mode can be focused and raised normally; Z-order is not the same thing as geometry.
- A floating window can cover magnetic windows. It need not be auto-relocated to keep everything visible.

Invariant M-01

An automatic layout pass may only change the position of Magnetic windows. It must never change the width or height of any window. An explicit user snap or direct resize is a different action.

04 / POSITIONING ENGINE

Magnetic arrangement behavior

Magnetism should create useful, readable foreground and background relationships without forcing every window into a grid.

When focus changes

- The newly focused window becomes the primary window of the active arrangement.
- Other Magnetic windows may reposition to expose as much useful visible area as possible around the active window.
- All window dimensions remain unchanged. Background windows may be partially obscured if there is insufficient space.
- Floating windows retain exact position and size; focusing them may still change native Z-order.
- Avoid needless moves: prefer stable existing positions when visibility is already acceptable.

Candidate placement priorities · proposed

Priority	Constraint or objective
Hard	Never change dimensions automatically; never move Floating windows.
Hard	Respect display usable bounds for requested snap rectangles; respect app sizing constraints for explicit resize.
High	Keep the active window visibly accessible and preserve mouse/keyboard usability.
High	Expose portions of background Magnetic windows where practical.
Medium	Minimize displacement from the previous stable arrangement and avoid unnecessary oscillation.
Low	Prefer aesthetically balanced gaps and alignment when no higher priority conflicts exist.

Freeze / defer layout while

- A native Space animation is taking place.
- The user is directly dragging or resizing a window.
- An app is presenting a transient dialog or an unusual window that should not be rearranged.

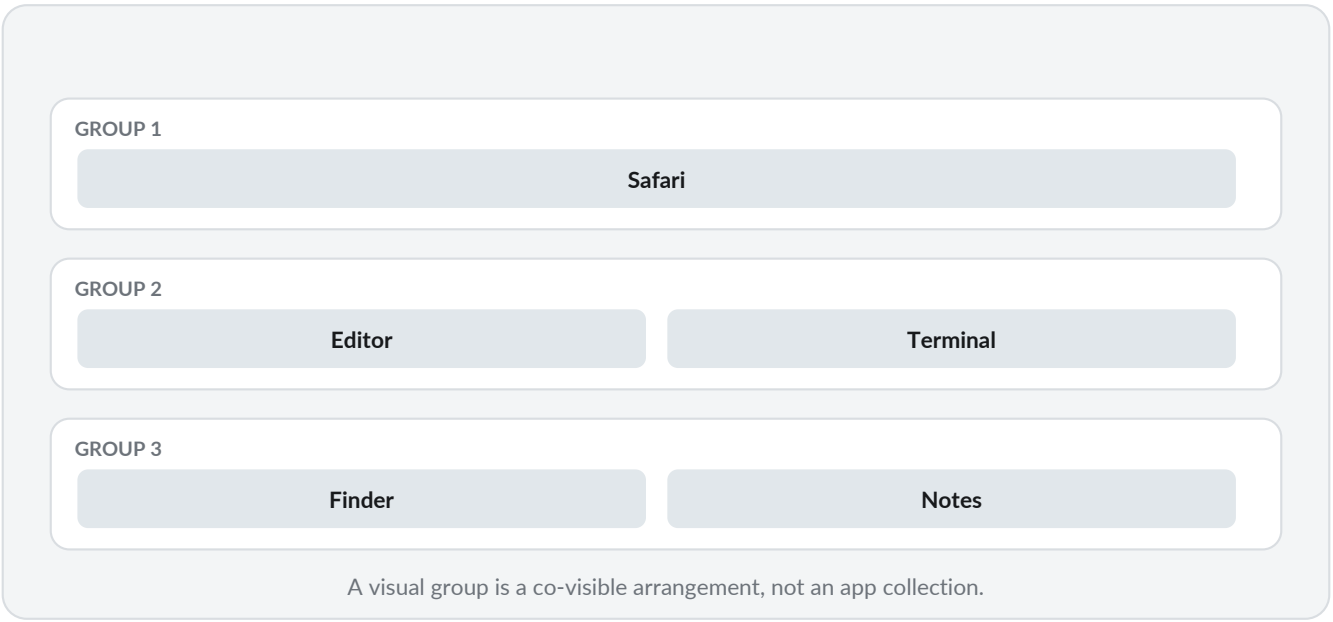
Important distinction

Magnetic position adjustment is not continuous physical scrolling. The v1 product is a focus-and-arrangement system, not an infinite off-screen vertical canvas.

05 / VISUAL NAVIGATION

Groups are the fast path

A side-by-side pair is a single visual group. Jumping to the next group must skip the other window in that pair.



A visual group is a co-visible arrangement, not an app collection.

Example

Suppose Group 1 contains Safari, Group 2 contains VS Code and Terminal in a split view, and Group 3 contains Finder. When Terminal is focused in Group 2:

Input	Expected result
Ctrl+Opt+Up / Down	Step to adjacent window in the ordered global stack; may first move between VS Code and Terminal.
Ctrl+Opt+Shift+Up	Jump directly to a member of Group 1, skipping any remaining Group 2 windows.
Ctrl+Opt+Shift+Down	Jump directly to a member of Group 3 in one operation.
Ctrl+Opt+H/J/K/L	Focus a spatial neighbor within Group 2 only; no crossing into another group.

Group creation and updates

The shared model is agreed; the exact grouping gesture is not yet finalized. Likely inputs include explicit half/quarter snapping and native-feeling window dragging. Do not infer or rebuild groups purely because spare screen area exists. A group is a presentation relationship, not necessarily a permanent tiling container.

06 / CONTROLS

Final keyboard conventions

Use Control + Option as the central navigation modifier. All shortcuts should be user-configurable because macOS and third-party conflicts vary.

Keys	Behavior	Status
⌃⌥ (hold)	Show window navigation overlay; release either key to hide.	Agreed
⌃⌥↑ / ↓	Previous / next individual window in the global stack.	Agreed
⌃⌥← / →	Previous / next window belonging to the active app.	Agreed
⌃⌥H / J / K / L	Focus left / down / up / right neighbor inside current visual group.	Agreed
⌃⌥⇧↑ / ↓	Jump to previous / next visual group.	Agreed
⌃⌥⌘ + arrows	Explicit position + size snapping: full, half and quarter areas.	Agreed intent
⌃⌥Space	Toggle Magnetic and Floating for the focused window.	Working convention

Reserved macOS and excluded keys

Keys	Policy
⌃↑ / ↓	Preserved for native Mission Control / App Exposé; not group jumping.
⌘Tab	Leave native application switching untouched.
⌃← / →	Leave native Space switching shortcuts intact.
⌥M	Deferred: the user has a different concept; define later.
Pixel resize shortcuts	Not included; snapping and direct resize cover v1 needs.

Spatial navigation rule

H/J/K/L affects focus only, based on actual neighbor geometry. It must not reorder, magnetize, move or resize windows. If no appropriate neighbor exists in the requested direction, do nothing.

07 / OVERLAY

Hold-to-preview navigator

The visual navigator is a temporary, non-focus-stealing layer shown while both Control and Option are held.

Required behavior

- Show while both modifiers are held. Hide as soon as either is released.
- Depict the ordered window stack, indicate the active window, and distinguish Magnetic from Floating.
- Show that windows can belong to the same application without collapsing them into one item.
- Update selection and mode live as shortcuts are used.
- Do not consume ordinary application focus or shift real window geometry merely because the overlay is visible.
- Quick shortcuts must work immediately even if overlay rendering has not completed.

Presentation direction · proposed

Lightweight, contextual, optional in appearance

A restrained navigation HUD, perhaps a slim vertical stack near an unobtrusive screen edge. Avoid a large custom desktop panel. Reveal only when the modifier is intentionally held.

Open UI questions

- Does the overlay appear instantly or after a very short configurable hold delay?
- Should the overlay show a linear window list, miniatures, or compact visual group separators?
- Does modifier release merely dismiss the HUD, or is an uncommitted preview model desirable? Current working assumption: focus changes immediately.

Accessibility requirements

Use readable text labels, clear selection indicators and a high-contrast fallback; avoid depending exclusively on transparent thumbnails. The overlay must never trap keyboard focus.

08 / GEOMETRY COMMANDS

Snapping, not pixel resizing

The positioning shortcut family changes both position and size, but only through clear, useful predefined regions.



Agreed behaviors

- Ctrl+Opt+Cmd + arrow: move the focused window to a predefined region and resize to match that region.
- The available region is determined by the usable display area, excluding system-reserved areas where appropriate.
- Supported classes: full usable area; left/right halves; screen quarters. Other regions are not necessary for v1.
- This is not native fullscreen: snapping must not create a fullscreen Space.
- Apply to Magnetic and Floating windows. Explicit resize is permitted in both modes; automatic resize is permitted in neither.
- Do not introduce separate 10 px / 40 px / percentage resize bindings.

Directional mapping remains open

An earlier proposal suggested that repeating the same directional shortcut cycles between half and quarter configurations. The user approved the general snapping modifier and size categories, not the exact cycling algorithm. Define and usability-test the mapping before freezing the keymap.

Candidate mapping	Illustrative only
Up	Full usable area (first press).
Left / Right	Corresponding half; repeated press may cycle corresponding quarters.
Down	Bottom half or restore, depending on finalized state machine.

09 / GESTURE MAP

Native trackpad conventions

Gesture priority is subordinate to system integrity: keep ordinary scrolling and native system gestures working.

Gesture	Target action	Boundary
3 fingers up / down	Jump to previous / next visual group.	One gesture = one group.
3 fingers left / right	Switch windows of the active application.	Does not switch Spaces.
4 fingers left / right	Native macOS Space switching.	Handled by macOS.
4 fingers up / down	Native Mission Control / App Exposé.	Handled by macOS.
2-finger scroll	Normal scrolling within applications.	Never globally hijack.

Technical and UX constraints

- macOS trackpad assignments can vary by user settings; recognize or guide around conflicts rather than silently override them.
- Independent global three-finger gesture recognition may require undocumented APIs or additional permissions; feasibility is not guaranteed.
- Horizontal three-finger gestures must not steal text-editing, application or system behaviors unexpectedly.
- Keyboard navigation remains fully functional if advanced gesture support is unavailable or disabled.

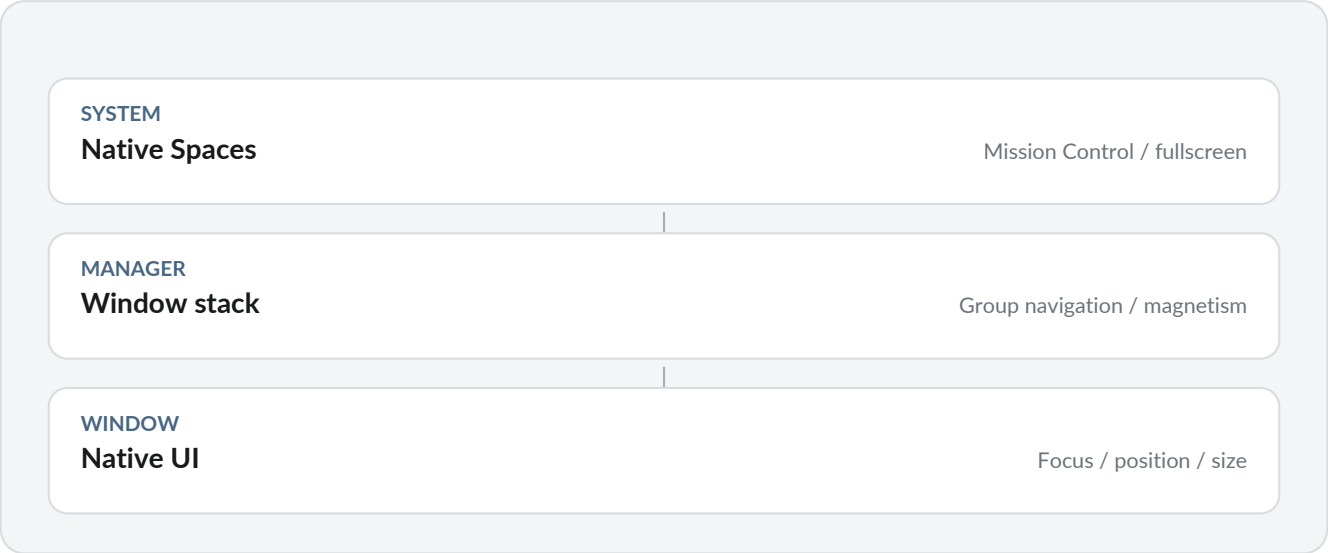
System shortcut conflict explicitly resolved

Control+Up and Control+Down remain reserved for the native macOS system. Group navigation uses Control+Option+Shift+Up/Down and the three-finger vertical gesture.

10 / MACOS INTEGRATION

Native Spaces remain authoritative

The manager is an addition to macOS window behavior, not a replacement desktop or a compositor.



System-owned behaviors

- Mission Control, App Exposé, actual Spaces, application activation, Dock and macOS fullscreen stay system-owned.
- Maintain separate management state for each recognized native Space (and display where applicable).
- On Space change, reconcile window membership and restore relevant focus/layout state without fighting the native transition.
- Dialogs, transient tool windows, menus and fullscreen windows require exception-aware handling.
- When the manager exits or encounters an error, do not leave application windows inaccessible.

Platform reality

Public Accessibility APIs enable manipulation of many windows, but do not expose a complete, stable external window compositor or comprehensive public Spaces management interface. Global gesture recognition and manipulation of off-screen windows are also constrained. The app must treat these as feasibility risks, not promised capabilities.

Open Space questions

- How to identify native Space membership reliably without depending on fragile assumptions?
- Whether movement between Spaces can remain native and stable; no v1 shortcut is locked for this.
- What happens with multiple monitors, separate Spaces per display, and monitor disconnection?
- How does Mission Control present Magnetic windows shifted to expose background windows?

11 / ENGINEERING ARCHITECTURE

Rust core, thin macOS adapter

Prefer a low-overhead event-driven native process. Keep the window and group model portable even when OS integrations are not.

Component	Responsibility
Rust domain core	Window stack, group membership, focus transitions, per-window mode, snap commands and layout planning.
macOS adapter	Window discovery, accessibility actions, input events, display geometry, Spaces information where possible.
Position solver	Auto-position Magnetic windows under fixed-size constraints; optimize visibility and movement stability.
Input coordinator	Modifier overlay state, keyboard shortcut routing, gesture commands and conflict handling.
Overlay renderer	Minimal, non-activating, native-looking HUD; Swift/AppKit/SwiftUI acceptable for thin presentation.
State + recovery	Restore stable user-owned rectangles and per-Space state; handle destroyed/recreated window identifiers.

Suggested internal types

Canonical state model

```
SpaceState { id, display_id, groups, window_order, focused_window }
WindowState { id, app_id, group_id, mode, frame, last_floating_frame }
VisualGroup { id, members, last_focused_window }
Command { FocusNext, FocusAppNext, FocusSpatial(dir),
JumpGroup(dir), ToggleMode, Snap(region) }
```

Performance requirements · directional

- No continuous window enumeration or high-frequency polling while idle.
- Coalesce event bursts; apply deterministic transactions with idempotent geometry updates.
- Avoid overlay activation, excessive accessibility calls and redundant window movement.
- Measure gesture latency and visual smoothness on real Mac hardware before claiming compositor-like behavior.

12 / VALIDATION

Acceptance scenarios for v1

These are behavioral tests, not promises that unverified macOS APIs are sufficient.

Scenario	Required outcome
Two windows share a left/right split	One group jump leaves both behind; individual-window navigation can select each separately.
Focus a different Magnetic window	Background Magnetic windows may reposition; no window dimension changes.
Focus a Floating window	The Floating rectangle stays unchanged; normal native focus/Z-order still works.
Hold Ctrl+Option	Overlay becomes visible and shows active window/mode; releasing either modifier hides it.
Press H/J/K/L in a group	Focus only moves to an eligible spatial neighbor within that group.
Use snapping shortcut	Chosen window moves to a valid region and resizes to full, half or quarter of usable area.
Use 3-finger vertical gesture	Moves to adjacent visual group, subject to reliable gesture support.
Switch native Spaces	Native transition remains intact; layout is not destructively reset.
Open modal / enter fullscreen	No incorrect magnetizing of a system-managed or transient window.
Exit / crash / restart	No stranded windows; restore enough state to resume safely.

Feasibility spikes before full product work

- Prototype a reliable Accessibility-based window enumeration/focus/movement loop.
- Measure Magnetic reflow of several windows without size changes or jitter.
- Test non-activating modifier-triggered overlay and keyboard event interception.
- Prototype global three-finger swipe detection while preserving native four-finger gestures.
- Validate actual native Spaces and Mission Control integration across macOS releases.

13 / DECISION LOG

Confirmed, proposed, deferred

This is the source of truth for future iterations; it prevents earlier brainstorming from being mistaken for an accepted requirement.

Status	Decision
CONFIRMED	Native macOS integration and simplicity take precedence over feature count.
CONFIRMED	Per-window Magnetic vs Floating; only Magnetic auto-repositions.
CONFIRMED	No automatic resizing of either mode; manual and snap resizing are explicit.
CONFIRMED	Groups are visual arrangements and can contain multiple different apps.
CONFIRMED	Ctrl+Opt modifier HUD; keyboard navigation families and H/J/K/L within a group.
CONFIRMED	Ctrl+Opt+Shift+Up/Down and 3-finger vertical gesture jump between visual groups.
CONFIRMED	Ctrl+Up/Down kept native; 4-finger system gestures retained; 2-finger scrolling untouched.
CONFIRMED	Ctrl+Opt+Cmd+arrows positions and snaps sizes to full/half/quarter.
WORKING	Ctrl+Opt+Space toggles mode; group entry restores last active member.
PROPOSED	Soft background exposure solver, minimal HUD visuals, native drag-based grouping.
OPEN	Precise snap cycling, group membership rules, gesture implementation and Space identification.
DEFERRED	Option+M, original offscreen infinite scrolling, custom overview, custom Spaces.

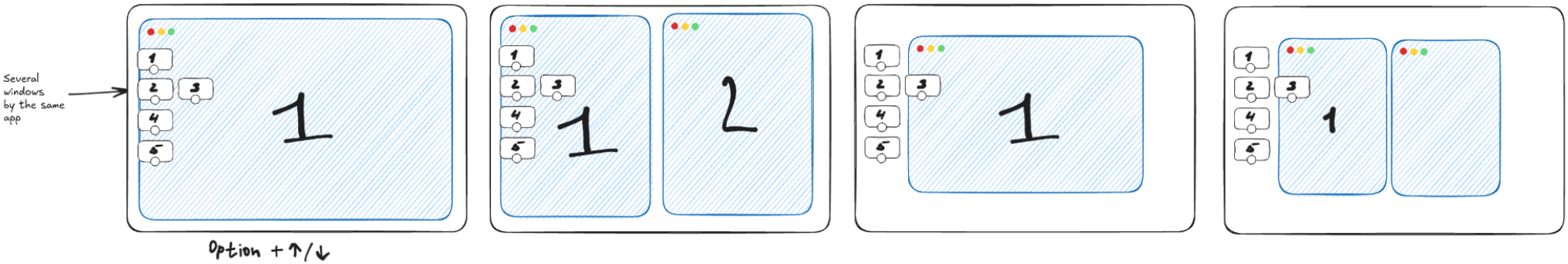
Glossary

Term	Meaning
Visual group	Co-visible set of one or more windows; navigation unit, not application unit.
Magnetic	Allows automatic position adjustment with fixed dimensions.
Floating	Keeps position and size independently of automatic layout updates.
Snap	Explicit position-and-size command for a standard screen region.
Window stack	Logical sequence used for individual-window navigation.

Next product design tasks: confirm snap-key state machine, define group membership rules, and prototype three-finger gestures.

APPENDIX / ORIGINAL CONCEPT SKETCH

Original user-provided drawing - earlier key combinations in the sketch have been superseded by the decision log.



1. When a user holds Option the visual view of the windows appear
2. When honldining option + clicking arrows up/down, the changing of window happens. Also should work with a trackpad gestures (swiping with 3 fingers up/down).
3. The active window should also be denoted in the visaul view
4. When the active app has multiple windows, using Option ->/ <- for changing the windows
5. Option + ctl + arrow up - make the window full size, Option + ctl + <-/ -> - make the window half of the screen, Option + M -> maximaze or minimaze the available view zone (the zone where a window can be)
6. The apps are not strictly alinged and sized, the use can change the size and position
When clicking Option + V -> there is a switch between floating mode and magnitized mode.
When magnitized, the user can change the position, but the window post aligned like by magnit in the correct closed posittion.
7. The windows that are behinde the active one should be seen, if there is a way thay can be seen.
Also, there should be an algorithm that will change the background (but still seen) windows in a way, that they can be seen in available space. Let's say: there are 4 windows, the active one is at the full size, than the user press Option + ctl + left/right (making a window in a half width), the window undernith must find the empty space, so in this case it ocupates the another half of the screen, If being on the one window (that is already half) clicks Option + ctl + up/down - make the window quater.
8. Option + shilft + arrows up/down - changing the posion in the Y-axis or in the stack (the order) if its the very end in the screen, Option + shift + left/right - moving window in the X-axis or between the Spaces if the very end.